

Technical Deployment Guide

Audience: a system administrator comfortable with git, Node.js, PostgreSQL, and Windows Server administration, who has **not** worked with this codebase before. This document takes you from `git clone` to a running production server **and** covers what needs to happen on the individual workstations (POS terminals, printer/scale machines, remote sites) that connect to it.

Status: this supersedes the scattered, partly stale deployment notes at the repo root (`DEPLOYMENT.md`, `INSTALLATION.md`, `SETUP.md`, `QUICK_DEPLOY.md`, `FRESH-INSTALL*.md`, `PRODUCTION-DEPLOYMENT-PROCEDURE.md`, etc.) for the core "get the server running" path. Those files haven't been deleted and may still hold useful historical context for specific past incidents, but where they disagree with this guide, **this guide reflects the actual current code** — every claim below was verified against the scripts and source, not just against prose in those files.

1. Architecture in one paragraph

This is a single Next.js 15 application with a **custom Node HTTP server** (`server.ts`, not `next start`) that also hosts a Socket.io server (used for real-time sync, the customer display, and the R710/Workstation local-agent protocols), a PostgreSQL database via Prisma, and an optional standalone Windows Service wrapper that keeps the whole thing running and can rebuild itself. There is no reverse proxy, load balancer, or container setup checked into the repo — the app is designed to be reached directly on its own port. Workstations are plain thin clients — a browser, and optionally a small local helper program (QZ Tray, the R710/Workstation agent, or an Electron kiosk shell) — nothing workstation-side runs its own copy of the application or database.

2. Prerequisites

Install these on the machine that will run the server, before touching the repo.

Software	Version	Notes
Node.js	20.x LTS	No <code>engines</code> field or <code>.nvmrc</code> is committed. <code>package.json</code> pins <code>next@15.5.4</code> , which requires Node ≥ 18.18 — Node 20 LTS is the safe, actively-supported choice. Ignore the "Node.js 16+" message printed by the installer script (<code>scripts/install/install.js</code>) — that check is stale against the actual Next.js requirement.
npm	ships with Node	No separate version pin found anywhere.
PostgreSQL	18.x	Required specifically if you'll use the automated Windows Service installer (§8) — it registers the app's service with a hard-coded Windows Service Control Manager dependency on a service literally named <code>postgresql-x64-18</code> (<code>windows-service/config.js</code>). If you install a different PostgreSQL major version, either install PG 18 as well or edit that <code>dependencies</code> array before running the service installer, or the service will fail to start. If you're not using the Windows Service installer, any reasonably current PostgreSQL (14+) is fine for the app itself.
Git	any current version	Used to clone the repo; also read at runtime by the Windows Service wrapper to detect when a rebuild is needed (it compares the currently checked-out commit to the commit the last build was made from).
openssl	any current version	Needed to generate the QZ Tray signing certificate (§7.2) — ships with Git for Windows, so usually already present.
mkcert	latest	Only needed if you'll serve the app over HTTPS (§7.1) — required in practice for QZ Tray printing to work in Chrome/Edge.

Native Node modules — platform build tools

Two dependencies compile native code: `bcrypt` (password hashing) and `canvas` (image/label rendering, barcode work). Both ship prebuilt binaries for common platform/Node-ABI combinations, so a plain `npm install` usually just works. If it doesn't (a `node-gyp rebuild` failure during `npm install`):

- **Windows:** install the "Desktop development with C++" workload via Visual Studio Build Tools, or run `npm install --global windows-build-tools` (older npm) / ensure Python 3 + the MSVC toolchain is on PATH.
- **Linux:** `sudo apt-get install -y build-essential libcairo2-dev libpango1.0-dev libjpeg-dev libgif-dev librsvg2-dev` (Debian/Ubuntu; adjust package manager for other distros) — these are `canvas`'s standard system dependencies, and are **not documented anywhere else in this repo**.

Is production Windows-only?

In practice, yes, even though the codebase isn't strictly Windows-only:

- `scripts/install/install-service.js` (the `sync` service installer, §8.1) does branch on `os.platform()` and will write a real `systemd` unit file on Linux.
- But the actual **self-healing production launcher** for the main app server (`windows-service/`, §8.2) uses the `node-windows` package and Windows Service Control Manager APIs — there is no Linux equivalent of it. On Linux you'd be running `node dist/server.js` directly under a plain `systemd` unit with no auto-rebuild-on-stale-commit behavior.

Everything in this guide works on Linux for running the app itself; §8.2 specifically (the hybrid auto-updating service) is Windows-only.

3. Get the code

```
git clone <your-repo-url> multi-business-multi-apps
cd multi-business-multi-apps
```

Pick the branch/tag you intend to deploy before continuing (`git checkout <branch>`).

4. Environment configuration

The repo ships a `.env.example` at the root — **it is not complete**. Copy it and then add the variables called out below that are missing from it entirely.

```
cp .env.example .env.local
```

4.1 Why `.env.local` and not `.env`

`server.ts` never calls `dotenv` itself — env loading happens implicitly the moment it constructs the Next.js app (`next({ dev, hostname, port })`), which uses Next's own built-in loader. Next's loader reads `.env.local` (and `.env`, `.env.production`, etc.) automatically, so **the running app process is fine with `.env.local`**.

The **Prisma CLI is not the app process** — run directly (`npx prisma migrate deploy`, `npx prisma studio`, ...), it only reads `.env` by default, never `.env.local`. This repo's convention is to keep real credentials in `.env.local` only, so raw `npx prisma ...` commands will fail to find `DATABASE_URL` unless you export it first. Two ways to deal with this:

- Use the project's own wrapper: `npm run db:deploy` (see `package.json`) — it loads `.env.local` via `dotenv` and then runs `prisma migrate deploy` with that environment, so you never hit this problem.
- Or export it manually in your shell before any raw `npx prisma ...` call:

```
# bash
export DATABASE_URL='postgresql://user:pass@localhost:5432/multi_business_db'
```

```
# PowerShell
$env:DATABASE_URL = 'postgresql://user:pass@localhost:5432/multi_business_db'
```

4.2 Required variables

These are enforced or required by code — the app will error or misbehave without them. **None of the three marked ⚠ below are present in `.env.example` at all** — you must add them yourself.

Variable	Required	Purpose	How to generate
DATABASE_URL	Yes	PostgreSQL connection string, read by Prisma	postgresql://<user>:<password>@<host>:5432/<database>
NEXTAUTH_SECRET	Yes	Session/token signing secret for NextAuth	node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"

Variable	Required	Purpose	How to generate
		(src/lib/auth.ts)	
ENCRYPTION_KEY ⚠️	Yes	AES-256 key used to encrypt sensitive stored fields (e.g. device admin passwords) — throws on first use if this is unset or not exactly 64 hex characters (32 bytes)	<code>node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"</code>
QZ_PRIVATE_KEY / QZ_CERTIFICATE ⚠️	Only if using QZ Tray printing	Signing keypair for QZ Tray print requests — see the full walkthrough in §7.2	<code>node scripts/generate-qz-cert.js</code> (requires <code>openssl</code> on PATH) — writes both values for you

A missing `ENCRYPTION_KEY` is a real, previously-hit failure mode: it surfaces as a generic `500` error with `"Encryption failed: ENCRYPTION_KEY environment variable is not set"` the first time any code path touches an encrypted field (e.g. registering a remote R710 device). Set it before first boot, not after something breaks.

4.3 Commonly-set optional variables

Variable	Default	Notes
PORT	8080	The app binds <code>0.0.0.0:\$PORT</code> — see §8.
NODE_ENV	—	Set to <code>production</code> for a real deployment.
NEXTAUTH_URL	unset	Deliberately left commented out in <code>.env.example</code> — leaving it unset allows login from both <code>localhost</code> and any LAN IP the server is reachable on. Only set it if you need to pin a single canonical URL.
FORCE_HTTPS	false	See §7.1.
CORS_ORIGINS	localhost + 192.168.0.0/16 + 10.0.0.0/8	Comma-separated allow-list. Adjust for your actual network if it doesn't fall in those private ranges.
LOG_LEVEL	info	<code>error</code> / <code>warn</code> / <code>info</code> / <code>debug</code> .
SYNC_*	—	Only relevant if this server participates in multi-branch peer sync — see <code>.env.example</code> 's SYNC section and <code>SERVICE_*</code> block. Irrelevant for a single-server deployment; leave as placeholders.

4.4 A separate set of variables used only by the installer

`scripts/install/install-database.js` provisions the database using its **own** set of env vars — it does **not** read `DATABASE_URL`:

```
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
POSTGRES_DB=multi_business_db
POSTGRES_SUPERUSER=postgres
```

If you use the automated installer (§8.1) with a custom database name/user, set these to match whatever you then put in `DATABASE_URL` — the two are not automatically kept in sync.

5. Database setup

You can do this manually (below) or via the automated installer (§8.1), which wraps the same steps. Manual steps, for full control:

5.1 Create the database and user

```
-- as the postgres superuser
CREATE DATABASE multi_business_db;
CREATE USER app_user WITH ENCRYPTED PASSWORD 'a-real-password';
GRANT ALL PRIVILEGES ON DATABASE multi_business_db TO app_user;
```

Put the resulting connection string in `.env.local` as `DATABASE_URL`.

5.2 Install dependencies and generate the Prisma client

```
npm install
npx prisma generate
```

5.3 Apply migrations

Always use `prisma migrate deploy` for a real deployment — never `prisma migrate dev`. `migrate dev` is a development-only command that can create a shadow database and prompt interactively; `migrate deploy` just applies the existing migration history non-interactively, which is what you want here. As of this branch there are **453 migration files** in `prisma/migrations/` — this will take a little while on a completely empty database.

```
npm run db:deploy
```

(`db:deploy` is the `.env.local`-aware wrapper described in §4.1; equivalent to `npx prisma migrate deploy` with `DATABASE_URL` exported manually.)

5.4 Seed reference data — required, not optional

A migrated-but-unseeded database is **not usable** — dropdowns, ID-number templates, phone/date format templates, job titles, compensation/benefit types, business categories, and permission templates all come from seed data, not from the schema itself. Run:

```
node scripts/production-setup.js --no-admin --ignore-missing-models
```

`--no-admin` is important here — see §5.5 for why.

Do **not** confuse this with the various `npm run seed:*` scripts (`seed:hardware`, `seed:clothing-bales`, `seed:customers`, etc.) — those are **demo/sample data** for testing and training environments, not required for a production deployment. Skip them unless you specifically want sample data.

5.5 Create the first admin user

```
node scripts/create-admin.js
```

This creates:

```
email:    admin@business.local
password: admin123
```

Change this password immediately after your first successful login — nothing in the app forces a password change on first login, and this is a well-known default.

Why `--no-admin` matters in §5.4: `scripts/production-setup.js` can *also* create its own admin user if run without `--no-admin` — a second, independent code path (`createSystemAdmin()`) that uses a different internal ID and a different, larger default permissions object than `scripts/create-admin.js` does. Both are functionally fine on their own (neither creates duplicates — both check for an existing user by email first), but running both against the same database is pointless and potentially confusing. **Stick to the sequence above:** `production-setup.js --no-admin` for reference data, then `create-admin.js` for the admin user, exactly as the automated installer does internally.

6. Build

```
npm run build
```

This runs, in order:

1. `prisma generate`
2. `next build` (with `NODE_OPTIONS=--max-old-space-size=8192` — the Next.js build genuinely needs a large heap on this codebase; don't strip that flag if you're scripting this yourself)
3. `npm run build:server` → `tsc --project tsconfig.server.json`, which compiles `server.ts` (and everything it transitively imports) to `dist/server.js`. **This step must succeed before `npm start` will work at all** — `npm start` runs `dist/server.js` directly, and that file only exists after this step.
4. `postbuild` (automatic): `scripts/mark-build-complete.js` (writes build markers, including the current git commit — this is what the Windows Service wrapper later checks for staleness) and then `scripts/build-agent.js` (builds the separate R710 local-agent `.exe/.zip` used for remote device support — **failure here is logged but does not fail the overall build**; if you don't need the R710 remote-agent feature you can ignore a warning from this step).

6.1 Verifying the build

```
ls dist/server.js      # must exist
ls .next/BUILD_ID      # must exist
```

If either is missing, `npm start` (§8) and the Windows Service wrapper (§8.2) will both fail.

7. Server-side certificates

There are **two entirely separate certificates** involved in a full deployment, generated and installed differently, for different purposes. Don't conflate them:

	TLS certificate (§7.1)	QZ Tray signing certificate (§7.2)
Purpose	Encrypts traffic between browsers and the app server (HTTPS)	Lets the app's print requests be trusted by QZ Tray on a workstation without a popup every time
Where it's generated	Anywhere with <code>mkcert</code> (often your own workstation, then copied to the server)	On the app server itself , via a script in this repo
Where it's installed	<code>certs/</code> folder at the app's repo root	<code>.env.local</code> (the key pair) and QZ Tray's own trust store on every workstation that prints via QZ Tray
Who needs to trust it	Every client browser (once each)	QZ Tray itself, on every printing workstation (once each)

7.1 TLS / HTTPS (browser ↔ server)

`server.ts` auto-detects HTTPS — there's no on/off flag. On startup it scans a `certs/` folder (git-ignored; does not exist on a fresh clone) for a `*.pem` file (excluding anything ending in `-key.pem` or named `rootCA.pem`) with a matching `<name>-key.pem`. If both are found, it starts HTTPS on the same port; otherwise it starts plain HTTP and logs that it did so. `NODE_ENV` has no effect on this decision.

Why you probably want this anyway

Chrome and Edge enforce **Private Network Access** restrictions that block a plain-HTTP page from talking to QZ Tray (which listens locally for print jobs) unless the page itself is served over HTTPS with a certificate the browser trusts. Firefox has no such restriction and needs none of this. If you don't use QZ Tray printing (e.g. every printer goes through the Windows Raw Printer path or an agent-relayed printer), you can skip this subsection entirely and run HTTP-only.

Installing `mkcert` itself

`mkcert` is not preinstalled anywhere and isn't a repo dependency — install it once on whichever machine will generate the certificate (usually the server itself, or any machine you trust to hold the private key briefly before copying it over).

Windows (PowerShell) — no package manager required, it's a single portable `.exe`:

```
New-Item -ItemType Directory -Force -Path "C:\mkcert" | Out-Null
Invoke-WebRequest -Uri "https://github.com/FiloSottile/mkcert/releases/download/v1.4.4/mkcert-v1.4.4-windows-
amd64.exe" -OutFile "C:\mkcert\mkcert.exe"

$env:Path += ";C:\mkcert"
[Environment]::SetEnvironmentVariable("Path", $env:Path, "User")
```

Close and reopen PowerShell, then confirm with `mkcert -version`. (Check <https://github.com/FiloSottile/mkcert/releases/latest> first in case a newer version exists — the URL above pins v1.4.4.) If you have Chocolatey or Scoop already, `choco install mkcert` / `scoop install mkcert` works too and skips the manual PATH step.

Linux (Debian/Ubuntu):

```
sudo apt-get update && sudo apt-get install -y libnss3-tools wget
wget -O mkcert https://github.com/FiloSottile/mkcert/releases/download/v1.4.4/mkcert-v1.4.4-linux-amd64
chmod +x mkcert
sudo mv mkcert /usr/local/bin/mkcert
mkcert -version
```

`libnss3-tools` is what lets `mkcert -install` also register the CA with Chrome/Chromium's own NSS trust store, not just the system one — skip it and Chrome-based browsers on that machine won't trust certificates `mkcert` issues. (Other distros: swap `apt-get` for your package manager — `nss-tools` on Fedora/RHEL, or use your distro's `mkcert` package directly if it has one, e.g. `sudo dnf install mkcert` / `sudo pacman -S mkcert`.)

Generating and installing the TLS certificate

```
# on the machine running the server, or anywhere with mkcert installed
mkcert -install # only needed once per machine, sets up mkcert's local root CA
mkcert <server-ip-1> <server-ip-2> localhost 127.0.0.1
```

This produces a cert + key pair (e.g. `192.168.0.108+3.pem` / `192.168.0.108+3-key.pem`) **in your current directory only** — see the ⚠️ callout below before assuming `rootCA.pem` is also freshly there. Then:

1. Create a `certs/` folder at the repo root (same level as `package.json`) and place the cert, key, and `rootCA.pem` inside it.
2. Restart the server: `npm run start` (or restart the Windows Service if that's how it's running — see §8.2).
3. Confirm the console (or that service's logs) shows `HTTPS enabled - certs loaded from ./certs/`.
4. The app is now reachable at `https://<server-ip>:8080` (or whatever port you configured).

Trusting this certificate on client machines is covered in §10.1 (workstation setup) — every machine that opens the app in Chrome/Edge needs to do this once.

⚠️ **List every IP/hostname the server will actually be reached by, in that one `mkcert` command — this is a real, previously-hit failure mode.** A cert generated for `192.168.0.108` will silently fail TLS hostname verification if the server is later reached at, say, `192.168.8.166` instead (a different NIC, a DHCP lease change, a new deployment target) — every browser tab and every local-agent socket connection to it fails with a generic, unhelpful error (browsers show a cert warning; a Node-based client like the R710/Workstation local agent just loops on a bare `websocket error` with no further detail). `rootCA.pem` being present and otherwise correct does not save you here — hostname/IP matching against the *leaf* cert's SAN list is checked independently of CA trust. If the server's reachable address ever changes, regenerate the leaf cert (`mkcert <new-ip> <old-ip-if-still-valid> localhost 127.0.0.1` **on the same machine that already holds the matching CA** — see the next warning if you're not sure that's true), replace the old cert+key pair in `certs/`, restart the server, and **re-pair** any local agents connected to it (revoking and re-pairing is what actually pushes a corrected `caCert` down to an agent — restarting the agent alone does not, since the bad/missing value is already saved in its own config).

⚠️ `mkcert <names>` **never copies `rootCA.pem` into your current directory — only the leaf cert + key.** This is a second, independent failure mode from the one above, and it's easy to trigger by accident: if you run `mkcert <names>` on a machine that has **never run `mkcert -install` before**, `mkcert` silently generates a **brand-new CA** on that machine (stored under its own per-user `mkcert -CAROOT`, e.g. `%LOCALAPPDATA%\mkcert` on Windows) and signs the new leaf cert with it — but nothing copies that new CA's `rootCA.pem` into `certs/`, so whatever `rootCA.pem` was already sitting there (from a previous setup, possibly generated on a completely different machine) is left in place, now signed by a CA that has nothing to do with the leaf cert actually being served. The server still starts and serves HTTPS fine — a browser that's already manually clicked through the warning, or that trusts the OLD CA from a previous setup, won't necessarily notice — but every local agent pairing will fail: the mint endpoint hands the agent the (wrong, stale) `rootCA.pem`, the agent's strict TLS validation

rejects the leaf cert it's actually shown, and the agent loops forever on a bare `websocket error` with no further detail, even after multiple re-pairs. To confirm and fix:

```
# confirm the mismatch – compare what the two files actually say
& "C:\Program Files\Git\usr\bin\openssl.exe" x509 -in certs\rootCA.pem -noout -subject -issuer
# if this org/CN doesn't match what the server is actually presenting on the wire, find the CA that
# actually signed the leaf cert:
mkcert -CAROOT
# then copy the correct one over:
Copy-Item "$(mkcert -CAROOT)\rootCA.pem" -Destination "certs\rootCA.pem" -Force
```

Restart the server, then **revoke and re-pair** every local agent connected to it — same reasoning as above, a re-pair is what actually delivers the corrected `caCert`. See §10.3's troubleshooting note for the equivalent workstation-side diagnostic steps (how to confirm this is actually the cause before assuming it).

7.2 QZ Tray signing certificate (generated on the server)

This is the certificate the user-facing docs refer to as "installing our own certificate" — it's separate from the TLS certificate above, generated **on the server** by a script already in this repo, and only matters if you're printing through QZ Tray (§10.2).

By default, QZ Tray shows the operator a trust popup on every single print job unless the request is cryptographically signed with a certificate QZ Tray has been told to trust. Generating your own certificate (rather than using QZ's bundled demo certificate) means that trust is set up once, deliberately, rather than relying on a certificate anyone else with the QZ demo files could also produce.

On the server, one-time:

```
node scripts/generate-qz-cert.js
```

This requires `openssl` on PATH (see §2) and:

1. Generates a 2048-bit RSA key pair and a self-signed certificate (10-year validity) with `openssl`.
2. Writes the public certificate to `certs/qz-certificate.pem`.
3. Prints two lines to the console — `QZ_PRIVATE_KEY=...` and `QZ_CERTIFICATE=...` (both base64-encoded) — **copy both into `.env.local`**. These are what the server uses to sign every print request it sends to a workstation's QZ Tray.
4. Restart the server (or the Windows Service — `npm run service:stop && npm run service:start`) so it picks up the new env vars.

At this point the server is signing its requests, but no workstation trusts the certificate yet — that half of the setup is per-workstation and covered in §10.2.

8. Running the server

There are three ways to run this in production, in increasing order of robustness. Pick one.

Option A — Plain foreground process (quick test only)

```
npm run build # if not already built
npm run start
```

This runs in the foreground and stops when the terminal closes. Fine for a smoke test; not a real production setup (no restart-on-crash, no auto-rebuild).

Option B — The automated installer (`npm run install:full`)

`node scripts/install/install.js` runs a **non-interactive**, six-step pipeline:

1. `preInstallationChecks()`
2. `installDependencies()` (`npm install`)
3. `installDatabase()` — everything in §5, automated (skip with `--skip-database`)
4. `installService()` — installs the **sync** service (see the naming warning in §8.1 below); skip with `--skip-service`
5. `postInstallationSetup()`
6. `finalVerification()`

```
npm run install:full
# or, to skip pieces you're handling manually:
npm run install:full -- --skip-database --skip-service
```

What this does NOT do: it does not run `npm run build` (§6), and it does not touch `certs/` (§7). You still need to run the build yourself and set up certificates if you want them, whether or not you use this installer.

8.1 ⚠️ Naming trap: two different things are both called "the service"

This repo has **two independently-installable Windows services, and their names are actively misleading:**

What you run	Windows service name	What it actually does
<code>scripts/install/install-service.js</code> (installed as part of <code>install:full</code> 's step 4, or standalone)	<code>multi-business-sync</code>	The peer-to-peer database sync background service. Optional. Irrelevant if you're running a single, non-syncing server.
<code>npm run service:install → node windows-service/force-install-hybrid.js</code>	<code>MultiBusinessSyncService</code> (display name "Multi-Business Sync Service")	Despite the name, this is the main application server launcher — see §8.2. It does not primarily do sync; the name is a legacy artifact.

Don't assume "I installed the service" from one of these covers the other. For a normal single-server production deployment, you want **§8.2's** service, not §8.1's.

8.2 Option C — The hybrid self-healing Windows Service (recommended for production)

```
npm run build:service # prepares windows-service/ assets
npm run service:install # must run from an elevated/Administrator shell
```

This registers a Windows service (internally named `MultiBusinessSyncService` — see the warning above) whose target script is `windows-service/service-wrapper-hybrid.js`. On every start, that wrapper:

1. Checks whether `dist/server.js` and `.next/BUILD_ID` exist, and compares the git commit recorded at last build time (written by `mark-build-complete.js`, §6) against the currently checked-out commit.
2. If the build is missing or stale relative to the current commit, **it runs `npm run build` itself** and streams the output, before proceeding.
3. Spawns `node -r tsconfig-paths/register dist/server.js` as a monitored child process, restarting it automatically on crash.

Practical implication: with this service installed, a normal deployment update is just:

```
git pull
# no need to manually rebuild or restart – restart the Windows service
# (Services console, or: net stop MultiBusinessSyncService && net start MultiBusinessSyncService)
# the wrapper detects the new commit and rebuilds automatically on that start
```

Prerequisites for this step:

- Run from an elevated (Administrator) shell — `node-windows` needs elevation to register a service with the Service Control Manager.
- PostgreSQL must be installed as the literal Windows service `postgresql-x64-18` (§2), or edit `windows-service/config.js`'s `dependencies` array to match your actual PostgreSQL service name before installing — otherwise Windows will refuse to start this service because its declared dependency doesn't exist.

Linux

There's no equivalent of §8.2's auto-rebuilding wrapper. Run the plain systemd unit `scripts/install/install-service.js` generates (`installUnixService()`), or write your own — either way, it starts `node dist/server.js` directly with no auto-build-on-stale-commit behavior, so **you are responsible for running `npm run build` yourself after every `git pull`** before restarting the service.

9. Networking

- The app binds `0.0.0.0:$PORT` (`server.ts`), default port **8080**, directly — not `127.0.0.1`, so it's reachable from other machines on the network as soon as it's running and the firewall allows it.
- **No reverse proxy config is checked into this repo** — no nginx, IIS, Caddy, or similar. If you want one in front of the app (for a stable hostname, additional TLS termination, rate limiting, etc.), that setup is entirely up to you; the app has no expectations about being behind one.
- `CORS_ORIGINS` (§4.3) needs to include whatever origin(s) the app is actually accessed from — the shipped default only covers `localhost`, `127.0.0.1`, and the `192.168.0.0/16` / `10.0.0.0/8` private ranges.
- Tailscale, if you see it mentioned anywhere in ops discussion for this project, is an operational choice for a specific site's remote access — it is not referenced anywhere in the application code or install scripts and is not a deployment requirement.

10. Workstation setup

Everything above is done once, on the server. This section is what needs to happen on each individual machine that will actually use the app — a POS terminal, a back-office PC, or a workstation with a printer/scale attached. Not every workstation needs every piece below — pick the ones relevant to how that particular machine is used.

10.1 Just browsing the app (every workstation)

1. Open the server's URL in a modern browser (Chrome, Edge, or Firefox) — `https://<server-ip>:8080` if TLS is set up (§7.1), otherwise `http://<server-ip>:8080`.
2. **If the server runs HTTPS with a self-signed (mkcert) certificate**, this browser needs to trust it once, or it'll show a security warning (and QZ Tray printing won't work at all in Chrome/Edge until this is done). Copy `rootCA.pem` from the server's `certs/` folder to the workstation (USB, shared folder, email — it's a public file, not a secret) first, then:
 - **Windows, automated:** alongside `rootCA.pem`, copy a small trust-installer script and run it — see the exact one-click pattern already used in `certs/README.md` (`setup-ssl.bat`) if you want to reuse that approach.
 - **Windows, manual:** `Win+R` → `certmgr.msc` → **Trusted Root Certification Authorities** → right-click **Certificates** → **All Tasks** → **Import** → select `rootCA.pem` → restart the browser.
 - **Linux (Debian/Ubuntu):**

```
sudo cp rootCA.pem /usr/local/share/ca-certificates/mkcert-rootCA.crt
sudo update-ca-certificates
```

This covers the system trust store (curl, wget, most non-Chromium apps). For Chrome/Chromium specifically, which uses its own NSS trust store, either install `libnss3-tools` and run `mkcert -install` locally using the *same* `rootCA-key.pem` (if you have it — not just the public `rootCA.pem`), or import it manually via `chrome://settings/certificates` → **Authorities** → **Import**. Firefox on Linux also uses its own store — `about:preferences#privacy` → **View Certificates** → **Authorities** → **Import**.

- **Firefox (any OS)** needs none of this for the *page itself* to load once you click through the one-time warning — it doesn't enforce the Private Network Access restriction that makes browser-level trust a hard requirement for QZ Tray in Chrome/Edge. Still recommended for a clean, warning-free experience either way.
3. Log in with a user account (the first admin account is created per §5.5 — change its password immediately if you haven't already).

10.2 Printing via QZ Tray (workstations with a locally-attached printer, using the QZ Tray path)

1. Install **Java 8+** if not already present (<https://java.com>) — QZ Tray requires it.
2. Install **QZ Tray** from <https://qz.io/download/> and leave it running (it starts with Windows by default, and sits in the system tray).
3. Make sure the printer itself is installed as a normal Windows printer first (see §10.4 if it isn't yet).
4. **Trust the server's signing certificate** (the one generated in §7.2, not the TLS one) — this is the "install our own certificate" step:
 - Copy `certs/qz-certificate.pem` from the server to this workstation (USB, shared folder, email — it's a public certificate, not a secret).
 - Right-click the QZ Tray icon in the system tray → **Advanced** → **Site Manager** → **Add Certificate** → select `qz-certificate.pem`.
 - From then on, print requests signed by the server with the matching `QZ_PRIVATE_KEY/QZ_CERTIFICATE` (§7.2) are trusted silently — no per-job popup.
5. In the app's POS receipt preview modal, the printer dropdown should now list this workstation's locally-installed printers (detected by QZ Tray). If it instead says "QZ Tray not detected," confirm QZ Tray is actually running in the system tray.

If you skip generating your own certificate (§7.2) and just want something working quickly on a trusted LAN, QZ Tray ships a demo certificate (`demo-signing.crt/demo-signing.key`, inside its own install folder) that can be used the same way — but because it's the same demo cert everyone gets, it's not something to rely on beyond a quick test.

10.3 R710 remote WiFi devices, or a remote scale/printer (workstations that need the local agent)

Only needed for: (a) an R710 WiFi controller that isn't on the same local network as the app server, or (b) a scale/printer physically attached to a workstation the server can't reach directly. If neither applies to a given workstation, skip this.

1. From the relevant device's **Agent panel** in the app (R710 Portal → Devices → Agent, or Admin → Workstation Agents), download **r710-agent.zip** — the download link is always available there, not just during initial setup.
2. Unzip it on the workstation and run **r710-agent.exe**. No installer, no separate Node.js/runtime needed — it's a self-contained executable. A tray icon appears.
3. From the browser, **on that same workstation**, open the Agent panel again and click **Pair this machine**.
4. Right-click the tray icon → **Preferences** → **Start with Windows** so it survives a reboot without manual intervention.
5. One agent install can be paired to R710, a scale, a printer relay, or any combination, and even to more than one server at once — see [docs/user-guide.md](#)'s R710/Workstation Agent sections for the full day-to-day usage, pairing screenshots, and troubleshooting detail; this guide only covers that it needs installing.

Troubleshooting: agent pairs successfully but immediately loops on a bare **websocket error**

If the agent's console shows the connection state cycling **connecting** → **error** → repeat, forever, and this doesn't clear up even after revoking and re-pairing, suspect a TLS certificate mismatch on the server side (§7.1's second ⚠️ callout) rather than anything wrong on the workstation. The agent (unlike a browser) cannot be manually told to trust an untrusted certificate — a mismatch there fails silently and permanently, with no more specific error surfaced.

To confirm it's a cert issue and not something else (firewall, wrong port, DNS), from the **workstation**, in PowerShell, test the raw TLS handshake directly against the server, independent of the agent entirely:

```
"" | & "C:\Program Files\Git\usr\bin\openssl.exe" s_client -connect <server-ip>:8080 -servername <server-ip>
```

Look at the **issuer=** line in the output — that names the machine whose mkcert CA actually signed what's being served. Then compare that against the actual current **certs\rootCA.pem on the server** (not a copy sitting on this or any other workstation — a stale local copy of that file is a common way to mislead this exact check):

```
& "C:\Program Files\Git\usr\bin\openssl.exe" x509 -in "certs\rootCA.pem" -noout -subject -issuer
```

If the **issuer=** from the live TLS handshake doesn't match the **subject=/issuer=** of the server's own **certs\rootCA.pem**, that confirms the exact scenario in §7.1's **mkcert gotcha** — fix it there (on the server), then come back and re-pair this agent. Restarting the agent alone will never fix this; the wrong **caCert** is already saved in its own pairing file, and only a fresh pairing (Revoke Pairing → Pair this machine again, no re-download needed) delivers a corrected one.

10.4 Receipt printer drivers (workstations printing via a "regular Windows printer," not QZ Tray/agent-relayed ESC/POS)

If a thermal printer needs to show up as a normal installed Windows printer (rather than being addressed directly over ESC/POS via QZ Tray or the local agent), it needs a Windows driver first. This repo has device-specific walkthroughs already:

- **INSTALL_EPSON_TM-T20III_DRIVER.md** (repo root) — EPSON TM-T20III
- **INSTALL_RONGTA_DRIVER.md** (repo root) — RONGTA 80mm series

Both boil down to: install the manufacturer's Windows driver (or let Windows auto-detect it under **Settings** → **Devices & Printers** → **Add a printer**), confirm it appears in **Devices and Printers**, then select it from the app's printer configuration (Admin → Network Printers, or the POS receipt preview modal, depending on connection mode).

10.5 Electron dual-monitor POS kiosk (optional, customer-facing display setups only)

If a POS terminal needs a second, customer-facing monitor showing a live order/cart display in kiosk mode (no browser chrome, can't be accidentally closed), that workstation can run the bundled Electron shell instead of a plain browser tab, from **electron/** in this repo:

```
cd electron
npm install
npm start          # auto-detects POS type + opens customer display on the second monitor
# or a specific POS type:
npm run start:restaurant
npm run start:grocery
npm run start:hardware
npm run start:clothing
```

This is purely a client-side convenience wrapper around the same app pages — it still connects to the same server URL and needs the same certificate trust as §10.1. Most workstations don't need this; it's only for a physical dual-monitor customer-display setup. See [electron/README.md](#) for packaging it as a standalone installable app rather than running from source.

11. Backup

```
npm run backup:database
```

wraps [scripts/backup-database.js](#). There isn't a separate, dedicated backup/restore reference document in this repo at the time of writing — if you need more detail than the script itself provides, read [scripts/backup-database.js](#) directly.

12. Post-deployment checklist

Server:

- ☐ `ENCRYPTION_KEY` set (64 hex chars) — §4.2
- ☐ `NEXTAUTH_SECRET` set to a real generated value, not the placeholder in `.env.example`
- ☐ Logged in as `admin@business.local` and **changed the default password**
- ☐ `npm run db:deploy` completed with no errors (453+ migrations applied)
- ☐ `node scripts/production-setup.js --no-admin --ignore-missing-models` completed (reference data seeded)
- ☐ `npm run build` completed, `dist/server.js` and `.next/BUILD_ID` both exist
- ☐ Decided TLS vs HTTP (§7.1) and, if TLS, confirmed the `HTTPS enabled` log line
- ☐ If using QZ Tray anywhere: generated the server's own QZ signing certificate (§7.2) rather than relying on QZ's demo cert long-term
- ☐ Chosen and installed one of the three run options in §8 — for real production, that's Option C
- ☐ Confirmed the PostgreSQL Windows service name matches what `windows-service/config.js` expects, if using Option C
- ☐ `CORS_ORIGINS` covers however this server will actually be reached
- ☐ A backup (`npm run backup:database`) has been taken at least once and you know where it writes to

Each workstation:

- ☐ Browser can reach the server and (if TLS) trusts its `rootCA.pem` (§10.1)
- ☐ QZ Tray installed + the server's `qz-certificate.pem` added via Site Manager, for any workstation printing that way (§10.2)
- ☐ Printer driver installed and visible in Windows Devices & Printers, for any printer that isn't QZ/agent-relayed (§10.4)
- ☐ R710/Workstation local agent downloaded, running, and paired — for remote R710 devices or a workstation-attached scale/printer (§10.3), with "Start with Windows" enabled
- ☐ Electron kiosk set up, only if that workstation drives a customer-facing second monitor (§10.5)

13. Known rough edges (worth knowing about, not necessarily worth fixing before you deploy)

- `scripts/install/install.js`'s Node version check says "16+" — ignore it; the real requirement (via Next.js 15) is Node ≥ 18.18 . Use Node 20 LTS.
- If `scripts/create-admin.js` is ever deleted or renamed, `install-database.js` falls back to a `createBasicAdmin()` path that hashes the password with plain SHA-256 (not bcrypt) and writes to a field name that doesn't match the current Prisma schema. This fallback is not currently reachable in a normal deployment — just don't delete `scripts/create-admin.js`.
- `.env.example` is missing `ENCRYPTION_KEY` and `QZ_PRIVATE_KEY/QZ_CERTIFICATE` entirely (§4.2) — don't assume it's a complete list of what the app needs.